

Week 8 Applied Sheet

(Solutions)

Useful advice: The following solutions pertain to the theoretical problems given in the applied classes. You are strongly advised to attempt the problems thoroughly before looking at these solutions. Simply reading the solutions without thinking about the problems will rob you of the practice required to be able to solve complicated problems on your own. You will perform poorly on the exam if you simply attempt to memorise solutions to these problems. Thinking about a problem, even if you do not solve it, will greatly increase your understanding of the underlying concepts. Solutions are typically not provided for Python implementation questions. In some cases, pseudocode may be provided where it illustrates a particular useful concept.

Problems

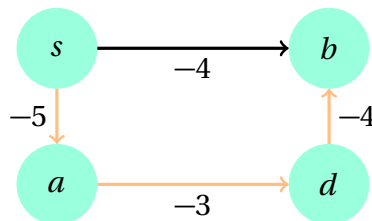
Problem 1. Consider the following algorithm for single-source shortest paths on a graph with negative weights:

- Find the minimum weight edge in the graph, say it has weight w
- Subtract w from the weight of every edge in the graph. The graph now has no negative weights
- Run Dijkstra's algorithm on the modified graph
- Add w back to the weight of the edges and compute the lengths of the resulting shortest paths

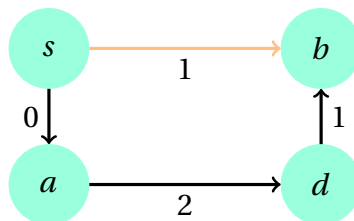
Prove by giving a counterexample that this algorithm is incorrect.

Solution

The problem with this algorithm is that it changes the length of paths with more edges a larger amount than it changes the lengths of paths with fewer edges, since it adds a constant amount to every edge weight. A good graph to break this would therefore be one where a shortest path has more edges than an alternative.



In this case, the shortest path from s to b is $s \rightarrow a \rightarrow d \rightarrow b$ with length -12 . If we subtract -5 from every edge weight in the graph, then it becomes the following, where the shortest path is now $s \rightarrow b$, so we get the wrong answer.



Problem 2. Describe an algorithm that given a directed connected graph $G = (V, E)$ determines whether or not G contains a negative cycle. Your algorithm should run in $O(|V| \cdot |E|)$ time.

Solution

Remember that the Bellman-Ford algorithm can detect whether there is a negative cycle that is reachable from the source vertex. The obvious tempting solution is to just run Bellman-Ford from every possible source vertex and see whether a negative cycle is detected. This would be very slow though, taking $O(|V|^2 \cdot |E|)$ time. Instead, we would rather run just one Bellman-Ford, but how can we be sure that we can definitely reach the negative cycle if one exists? The easiest way to ensure this is to simply add a new vertex to the graph, and add an edge from that vertex to every other vertex in the graph. Running Bellman-Ford on this vertex will then definitely visit every vertex, and hence will definitely detect a negative cycle if one is present. Since we only need to run Bellman-Ford once, this algorithm takes $O(|V| \cdot |E|)$ time.

Note that if the original graph $G = (V, E)$ was not connected, then we could still use the same strategy, but the running time would be $O(|V| \cdot |E| + |V|^2)$ to account for cases such as a graph with $|V|$ isolated nodes (i.e., $|E| = 0$). Note also that $O(|V| \cdot |E| + |V|^2)$ simplifies to $O(|V| \cdot |E|)$ for connected graphs.

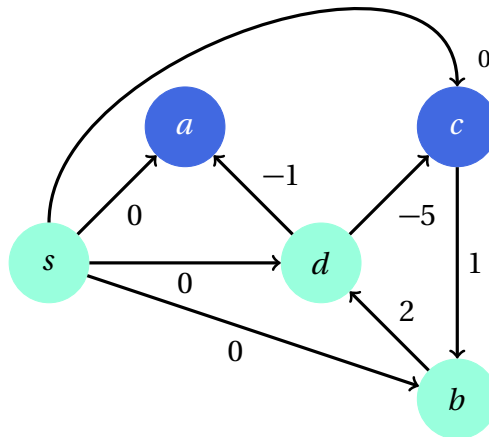
Problem 3. Given a connected graph $G = (V, E)$ that contains a negative weight cycle, give an algorithm to determine the vertices of one such cycle. Your algorithm should run in $O(|V| \cdot |E|)$ time.

Solution

Recall from Problem 2 that we can use Bellman-Ford to detect the presence of a negative cycle by adding a new source vertex that connects to all others. Let's start by doing this. Remember that when Bellman-Ford terminates, the predecessors will all form a shortest path tree, except for the vertices reachable from a negative cycle.

Arbitrarily select a vertex which had its distance updated during the $|V|$ -th iteration. This vertex may not necessarily be **in** a negative cycle, as it could be on a path that simply travels through one and then leaves it. In order to get to the negative cycle that caused this, we should travel backwards along the predecessors until we see a repeated node u . The negative cycle consists of u together with all nodes that appeared between the first and second occurrences of u .

The graph below provides an example where a vertex which is not part of a negative cycle has its distance updated during the $|V|$ -th iteration. This highlights how not all updated vertices are necessarily part of a negative cycle. Also note how not **all** vertices that are part of the negative cycle have their distances updated in the $|V|$ -th iteration (in this example, d and b).



Iteration #	s	a	b	c	d
$ V - 1$	0	-1	-4	-5	-2
$ V $	0	-3	-4	-7	-2

If a graph does indeed have a negative cycle, at least one of the vertices on the cycle will always update every iteration, but potentially also vertices that are reachable from the vertices in the negative cycle via offshoot branches, as those distance updates will ripple down each branch. The number of these extra detected vertices will depend on the specific relaxation order of edges. Therefore, we must follow the backtracking process outlined above to guarantee we find one negative cycle.

For the backtracking process we can keep a “visited” array to detect what is the first vertex to be visited twice. If we do not want to use extra auxiliary space, we can instead initially travel backwards $|V| - 1$ vertices to guarantee that one gets to some vertex u in a negative cycle, and from there travel the negative cycle once until getting back to u .

Problem 4. Improve the Floyd-Warshall algorithm so that you can also reconstruct the shortest paths in addition to the distances. Your improvement should not worsen the time complexity of Floyd-Warshall, and you should be able to reconstruct a path of length ℓ in $O(\ell)$ time.

Solution

There are many approaches to make this work.

Solution 1: Track the intermediate vertex

One way to keep track of the path information is to record, for each pair of vertices u, v , which intermediate vertex was optimal for going between them. This can be tracked during the algorithm like so.

```

1: function FLOYD_WARSHALL( $G = (V, E)$ )
2:   Set  $\text{dist}[1..n][1..n] = \infty$ 
3:   Set  $\text{dist}[v][v] = 0$  for all vertices  $v$ 
4:   Set  $\text{dist}[u][v] = w(u,v)$  for all edges  $e = (u,v)$  in  $E$ 
5:   Set  $\text{mid}[1..n][1..n] = \text{null}$                                 //  $\text{mid}[u][v] = \text{optimal intermediate vertex}$ 
6:   for each vertex  $k = 1$  to  $n$  do
7:     for each vertex  $u = 1$  to  $n$  do
8:       for each vertex  $v = 1$  to  $n$  do
9:         if  $\text{dist}[u][k] + \text{dist}[k][v] < \text{dist}[u][v]$  then
10:            $\text{dist}[u][v] = \text{dist}[u][k] + \text{dist}[k][v]$ 
11:            $\text{mid}[u][v] = k$ 
12:   return  $\text{dist}[1..n][1..n], \text{mid}[1..n][1..n]$ 

```

For each pair of nodes u and v that have a finite distance, to reconstruct the path, we then need to do a sort of divide-and-conquer style path reconstruction.

```

1: function GET_PATH( $u, v$ )
2:   if  $\text{mid}[u][v] = \text{null}$  then
3:     return  $[u, v]$ 
4:   else
5:      $\text{left} = \text{GET\_PATH}(u, \text{mid}[u][v])$ 
6:      $\text{right} = \text{GET\_PATH}(\text{mid}[u][v], v)$ 
7:     return  $\text{left.pop\_back()} + \text{right}$                                 // Remove the duplicate from the middle

```

Note that this is only efficient if the concatenation on line 7 is $O(1)$, which means that `left` and `right`

would need to be implemented as linked lists. **Solution 2: Track the successor**

To avoid having to use linked lists, we could take a different approach. We will remember, for each pair u, v , the first vertex on a shortest path from u to v (i.e the successor of u on the shortest path to v). To maintain this, if we update the pair u, v and decide that vertex k is the new best intermediate vertex, then the first vertex on a shortest path from u to v is just the first vertex on a shortest path from u to k . We can implement this as follows.

```

1: function FLOYD_WARSHALL( $G = (V, E)$ )
2:   Set  $\text{dist}[1..n][1..n] = \infty$ 
3:   Set  $\text{dist}[v][v] = 0$  for all vertices  $v$ 
4:   Set  $\text{dist}[u][v] = w(u,v)$  for all edges  $e = (u,v)$  in  $E$ 
5:   Set  $\text{succ}[1..n][1..n] = \text{null}$  //  $\text{succ}[u][v] = \text{successor of } u \text{ on shortest path to } v$ 
6:   Set  $\text{succ}[u][v] = v$  for all edges  $e = (u,v)$  in  $E$ 
7:   for each vertex  $k = 1$  to  $n$  do
8:     for each vertex  $u = 1$  to  $n$  do
9:       for each vertex  $v = 1$  to  $n$  do
10:        if  $\text{dist}[u][k] + \text{dist}[k][v] < \text{dist}[u][v]$  then
11:           $\text{dist}[u][v] = \text{dist}[u][k] + \text{dist}[k][v]$ 
12:           $\text{succ}[u][v] = \text{succ}[u][k]$ 
13:   return  $\text{dist}[1..n][1..n], \text{succ}[1..n][1..n]$ 

```

Reconstructing the paths is now easier as we do not need any fancy recursion or linked lists, since we only need to append one vertex at a time rather than concatenating two lists.

```

1: function GET_PATH( $u, v$ )
2:   Set  $\text{path} = [u]$ 
3:   while  $u \neq v$  do
4:      $u = \text{succ}[u][v]$ 
5:      $\text{path.append}(u)$ 

```

In both cases, we add only a constant amount of overhead to the algorithm so we do not worsen the complexity. We can also reconstruct paths of length ℓ in $O(\ell)$ time in both cases.

Problem 5. Add a post-processing step to Floyd-Warshall that sets $\text{dist}[u][v] = -\infty$ if there is an arbitrarily short path between vertex u and vertex v , i.e. if u can travel to v via a negative weight cycle. Your post processing should run in $O(|V|^3)$ time or better, i.e. it should not worsen the complexity of the overall algorithm.

Solution

Recall that the Floyd-Warshall algorithm can detect the presence of negative cycles by looking at the distances of vertices to themselves. If a vertex can reach itself with a negative distance, then it must be contained in a negative cycle. We can therefore post process the graph by checking each intermediate vertex k that is part of a negative cycle, and then setting the lengths of all paths $u \rightsquigarrow v$ such that u can reach k and k can reach v to $-\infty$. This process takes $O(|V|^3)$. See the pseudocode below.

```

1: function FLOYD_WARSHALL_POSTPROCESS( $G = (V, E), \text{dist}[1..n]$ )
2:   for each vertex  $k = 1$  to  $n$  do
3:     if  $\text{dist}[k][k] < 0$  then
4:       for each vertex  $u = 1$  to  $n$  do
5:         for each vertex  $v = 1$  to  $n$  do
6:           if  $\text{dist}[u][k] + \text{dist}[k][v] < \infty$  then
7:              $\text{dist}[u][v] = -\infty$ 

```

Problem 6. Arbitrage is the process of exploiting conversion rates between commodities to make a profit, and it may occur over many steps. For example, one could purchase US dollars, convert it into British pounds, and then back into Australian dollars. If the prices were right, this could result in a profit. Given a list of n currencies and the best available conversion rate between some pairs of currencies (i.e., how much of the first currency you need to spend to get 1 unit of the second currency), devise an algorithm that determines whether arbitrage is possible, i.e. whether or not you could make a profit. Your algorithm should run in $O(n^3)$ where n is the number of currencies.

Solution

Let's imagine this as a shortest path problem. We wish to begin at one currency and walk through a path containing some other currencies before arriving back at our initial currency with a net profit. Making a profit means that the net conversion rate between a currency and itself should be less than 1. Note that if we convert between currencies whose exchange rates are $r_1, r_2, \dots, r_k$, then the final conversion rate is $r_1 \times r_2 \times \dots \times r_k$. One way to model this problem is to therefore create a graph on n vertices where each vertex represents a currency. We add a directed edge between a pair of vertices with the exchange rate as the weight. We could then modify the Floyd-Warshall algorithm to compute the product of the weights rather than the sum, and it would then find for us the best conversion rate between any pair of currencies. Arbitrage is possible if a currency can be converted into itself at a rate of less than 1, i.e. if we find an entry on the diagonal of the distance matrix that is less than 1.

An alternate solution that does not require us to modify Floyd-Warshall is to create the same graph but use the logarithms of the exchange rates as the edge weights. We can then run the ordinary Floyd-Warshall algorithm. This works because $\log(r_1) + \log(r_2) = \log(r_1 \times r_2)$. Since $\log(r) < 0$ if and only if $r < 1$, we know that arbitrage is possible if and only if the graph contained a negative cycle, i.e. if a vertex has a negative distance to itself.

Problem 7. In this problem, we will derive Johnson's algorithm, a fast algorithm for the all-pairs shortest path problem on sparse graphs $G = (V, E)$ with negative weights assuming that the graph has no negative cycles. The key ingredient of the algorithm is based on the idea of *potentials*.

- (a) Let $d[v] : v \in V$ be the distances to each vertex $v \in V$ from an arbitrary source vertex. Prove that for each edge $(u, v) \in E$

$$d[u] - d[v] + w(u, v) \geq 0.$$

We define the quantity $p[u] = d[u]$ as the *potential* of the vertex u .

- (b) Give a simple algorithm to compute a set of valid potentials. Potentials should not be infinity for any vertex.

Suppose now that we modify the weight of every edge (u, v) in the graph to

$$w'(u, v) = p[u] - p[v] + w(u, v).$$

- (c) Prove that a shortest path in the modified graph was also a shortest path in the original, unmodified graph.
(d) Deduce that we can solve the all-pairs shortest path problem on sparse graphs with negative weights in $O(|V|^2 \cdot \log|V|)$ time.

Solution

- (a) This is a direct result of the triangle inequality. The triangle inequality states that

$$d[u] + w(u, v) \geq d[v],$$

which must be true since if this quantity were less than $d[v]$, then the path $s \rightsquigarrow u \rightarrow v$ would be shorter than the shortest path to v , a contradiction. Subtracting $d[v]$ from both sides of the triangle inequality yields the desired result.

- (b) A solution that almost works is to pick an arbitrary source vertex and run Bellman-Ford to compute

the shortest distances. This will fail though if the source we pick cannot reach every vertex since then some of the distances will be infinity. To correct this, we instead add a new vertex to the graph and connect it to every vertex with an edge weight of zero. We then run Bellman-Ford from this new vertex. By design, it is connected to every other vertex so none of the distances will be infinity. The resulting distances can be used as potentials.

- (c) Consider a path in the modified graph $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k$. Its total length will be

$$p[v_1] - p[v_2] + w(v_1, v_2) + p[v_2] - p[v_3] + w(v_2, v_3) + \dots + p[v_{k-1}] - p[v_k] + w(v_{k-1}, v_k).$$

Observe that except for the first and last, the potentials all cancel since they appear as a positive term and then as a negative term immediately after. This leaves us with

$$p[v_1] + w(v_1, v_2) + w(v_2, v_3) + \dots + w(v_{k-1}, v_k) - p[v_k].$$

Note that the only potentials left in the equation are of the first and last vertex, in other words, they do not depend on the path taken at all. The remaining terms simply constitute the path length of the path $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k$ in the original, unmodified graph, hence any shortest path in the modified graph is also a shortest path in the unmodified graph.

- (d) With all of these ingredients, we can derive *Johnson's algorithm*. First, we run Bellman-Ford from a new source vertex that connects to every other vertex in order to obtain a set of valid potentials. We then create the new graph by modifying the edge weights as above. Now since all of the edge weights are nonnegative by (a), we can use Dijkstra's algorithm to compute shortest paths from every possible source vertex. Finally we can recover the true shortest paths by subtracting away the potentials that we added. In sparse graphs one invocation of Bellman-Ford costs $O(|V|^2)$ and $|V|$ Dijkstra's costs $O(|V|^2 \cdot \log |V|)$, and hence the total cost is $O(|V|^2 \cdot \log |V|)$.

Supplementary Problems

Problem 8. Implement Bellman-Ford and test your code's correctness by solving the following problem on UVA Online Judge: https://uva.onlinejudge.org/index.php?option=onlinejudge&page=show_problem&problem=499.

Problem 9. Devise an algorithm for counting the number of paths between two given vertices s and t in a directed acyclic graph $G = (V, E)$ which runs in $O(|V| + |E|)$. Don't worry about the magnitude of the answer (i.e. assume that all arithmetic operations take constant time, despite the fact that the answer might be exponential in the input size.)

Solution

We will approach this using a dynamic programming algorithm. Suppose we are at some vertex u . Then the number of different ways in which we could get to t involves trying all of our different edges (u, v) and then counting the number of resulting paths from all of the v . So let us define the following subproblems.

$$\text{DP}[u] = \{\text{The number of paths from } u \text{ to } t\}.$$

The recurrence is then given by

$$\text{DP}[u] = \begin{cases} 1 & \text{if } u = t, \\ \sum_{v \in \text{adj}[u]} \text{DP}[v] & \text{otherwise.} \end{cases}$$

The answer is the value of $\text{DP}[s]$. The subproblems are dependent in a reverse topological order, since in order to compute $\text{DP}[u]$ we must know the value of all of u 's descendants.

```

1: function COUNT_PATHS( $G = (V, E)$ ,  $s$ ,  $t$ )
2:   Set  $\text{count}[1..n] = 0$ 
3:    $\text{count}[t] = 1$ 
4:   Set  $\text{order} = \text{reverse}(\text{topological\_sort}(G))$ 
5:   for each vertex  $u$  in order do
6:     for each edge  $(u, v)$  adjacent to  $u$  do
7:        $\text{count}[u] = \text{count}[u] + \text{count}[v]$ 
8:   return  $\text{count}[s]$ 

```

Problem 10. Consider a directed acyclic graph $G = (V, E)$ where each edge is labelled with a character from some finite alphabet A . Given a string S over the alphabet A , count the number of paths in G whose edge labels spell out the string S . Your algorithm should run in $O((|V| + |E|) \cdot n)$, where n is the length of the string S .

Solution

This problem can be solved very similarly to the ordinary path-counting problem on a DAG (Problem 9). We will use dynamic programming. Let us define the subproblems

$$\text{DP}[u, i] = \{\text{The number of paths from } u \text{ that spell out } S[i..n]\}$$

If we are at a particular vertex u and we want to write $S[i..n]$, then we need to find all of our outgoing edges (u, v) that are labelled $S[i]$, and then try to write $S[i + 1..n]$ from vertex v . A recurrence can therefore be written as follows.

$$\text{DP}[u, i] = \begin{cases} 1 & \text{if } i = n + 1, \\ \sum_{\substack{v \in \text{adj}[u] \\ \ell(u, v) = S[i]}} \text{DP}[v, i + 1] & \text{otherwise} \end{cases}$$

where $\ell(u, v)$ denotes the label of the edge (u, v) . The solution to the problem is then the sum of $\text{DP}[u, 1]$ for all vertices $u \in V$. We have $V \cdot n$ subproblems, and for each edge in the graph, it gets processed at most once per value of i , so the total time complexity is $O(|V| \cdot n + |E| \cdot n) = O((|V| + |E|) \cdot n)$ as required.

Problem 11. An improvement to the Bellman-Ford algorithm is the so-called “Shortest Paths Faster Algorithm”, or SPFA. SPFA works like Bellman-Ford, relaxing edges until no more improvements are made, but instead of relaxing every edge $|V| - 1$ times, we maintain a queue of vertices that have been relaxed. At each iteration, we take an item from the queue, relax each of its outgoing edges, and add any vertices whose distances changed to the queue if they are not in it already. The queue initially contains the source vertex s .

- Reason why SPFA is correct if the graph contains no negative-weight cycles.
- The algorithm described above will cycle forever in the presence of a negative-weight cycle. Add a simple condition to the algorithm to fix this and detect the presence of negative cycle if encountered.
- What is the worst-case time complexity of SPFA?

Solution

The proof of correctness for SPFA is pretty much exactly the same as for Bellman-Ford, so we will not write the whole thing formally. At the first iteration, all shortest paths consisting of at most 1 edge will be found by relaxing the edges out of s . Subsequently, all shortest paths of length at most two will be found when those vertices relax their outgoing edges, and so on. By induction, we can show that all shortest paths of length k will be relaxed after the shortest paths of length $k - 1$. SPFA essentially does the same thing as Bellman-Ford, but it avoids relaxing edges that will not make a difference.

In the presence of a negative weight cycle, SPFA will continue to add the vertices of the cycle to the queue

forever. To stop this, we enforce that each vertex can only be removed from the queue at most $n - 1$ times. If a vertex is removed n times, this implies that the shortest path to it contains $n + 1$ vertices since the queue always serves vertices in order of the number of edges on their shortest path, and in between multiple occurrences of the same vertex, the path length must increase.

Since we may remove each vertex from the queue n times, and each time we process every one of its edges, the amount of work done is at most $O(|V| \cdot |E|)$, the same as Bellman-Ford. Although this sounds bad, on average SPFA performs much better.

Problem 12. Describe how an instance of the unbounded knapsack problem can be converted into a corresponding directed acyclic graph. Which graph problem correctly models the unbounded knapsack problem on this graph?

Solution

Create a graph with $C + 1$ vertices, labelled 0 to C , where C is the capacity of the knapsack. For each item available with weight w and value v , create an edge from every vertex $c \geq w$ to the vertex $c - w$ with weight v . The solution to the unbounded knapsack problem is the critical path of this graph.

Problem 13. (Advanced) A problem closely related to the transitive closure problem mentioned in the lecture recordings is the *transitive reduction*. In a sense, the transitive reduction is the opposite of the transitive closure. It is a directed graph with the fewest possible edges that has the same reachability as the original graph. In other words, for all pairs of vertices u and v , there is a path between u and v in the transitive reduction if and only if there is a path between u and v in the original graph. Give an algorithm for computing the transitive reduction of an unweighted directed acyclic graph $G = (V, E)$. Your algorithm should run in $O(|V|^2 + |V| \cdot |E|)$ time.

Solution

As usual, we will assume that G is a simple graph. If it is not, we can simply remove duplicate edges and loops at the beginning since they contribute nothing to the reachability. Consider each edge (u, v) in the graph G . If (u, v) is the only way to travel from u to v , then clearly we must keep it. Otherwise, if there is some other way to get from u to v , since the graph is simple, there must be a path consisting of multiple edges that leads from u to v . If this is the case, removing (u, v) does not harm the reachability, and since the other vertices on the $u \rightsquigarrow v$ path must maintain their respective reachabilities, we should remove (u, v) , rather than removing anything on that path.

Given this, our algorithm is as follows. Let's use the critical path dynamic programming algorithm to compute the longest paths between every pair of vertices in the graph. Computing the longest paths that start at a particular vertex takes $O(|V| + |E|)$, so doing this from every vertex takes $O(|V|^2 + |V| \cdot |E|)$ time. Then, for each edge (u, v) , we decide to keep it if the longest path from u to v is length 1 (i.e. just this edge), otherwise we delete it. This algorithm takes $O(|V|^2 + |V| \cdot |E|)$ time in total.

This can be easily generalised to weighted directed acyclic graphs by considering weights 1 for all edges while computing the longest paths.